

# Arquitetura completa da Samsung Xclipse 940 — XO940

**Projeto:** Xclipse Open Project

**Alvo de referência:** Samsung SM-S721B, plataforma `s5e9945` / `erd9945` / `r12s`

**Hardware gráfico:** Samsung Xclipse 940, família MGFX 147

**Revisão documental:** 2026-09-18

**Natureza:** mapa arquitetural baseado em Device Tree, runtime, traces, inventários ELF, fontes inventariadas e resultados sanitizados

## 1. Como ler este documento

Este documento descreve a arquitetura observada em camadas. Cada camada informa quais objetos, relações ou eventos foram encontrados e qual é o limite da interpretação. A presença de uma biblioteca, símbolo, arquivo de firmware, nó Device Tree ou ioctl não é convertida automaticamente em uma afirmação de execução.

A legenda usada ao longo do documento é:

| Classe                      | Significado                                                                              |
|-----------------------------|------------------------------------------------------------------------------------------|
| <b>Confirmado</b>           | Observado diretamente em runtime, trace, ioctl, Device Tree ou saída reproduzida.        |
| <b>Confirmado por fonte</b> | Encontrado em fonte, manifesto ou artefato técnico e compatível com a captura.           |
| <b>Parcial</b>              | Parte da cadeia foi observada, mas falta uma etapa para uma conclusão fim a fim.         |
| <b>Indício</b>              | Símbolo, string, export ou caminho que orienta a arquitetura, sem execução independente. |
| <b>Negativo específico</b>  | Uma tentativa concreta falhou; o resultado não generaliza para todas as alternativas.    |
| <b>Hipótese</b>             | Relação de implementação ainda não demonstrada.                                          |

## 2. Mapa geral do SoC

SoC Exynos / Android

### Plataforma

- Device Tree
- power domains
- clocks / DVFS / IFPO / thermal / AFM
- IRQ / reset / reserved memory
- DMA heaps / BTS / SysMMU / IOMMU
- variantes de placa e revisão

### GPU Xclipse 940 / G3D

- sgpu@22200000
- SGPU kernel driver / DRM
- UAPI e ioctls
- GEM / BO / TTM / DMA-BUF
- VM / PTE / PDE / IOMMU
- context / BO\_LIST / CS / chunks
- scheduler / rings / IB / doorbell
- GFX / COMPUTE / SDMA
- firmware / reset / recovery
- fences / syncobj / dependencies
- renderD128 / renderD129 / card0 / card1

### Pilha Android gráfica

- SurfaceFlinger
- RenderThread / Gralloc / câmera
- libdrm\_sgpu.so
- vulkan.samsung.so
- libOpenCL.so / libSGPUOpenCL.so
- linker namespaces
- manifests / HAL
- SELinux / permissões / storage noexec

### NPU Exynos / NNAPI / ENN

- IDevice/enn
- HAL neuralnetworks-service-enn
- runtime / compiler / NCP / CMDQ
- npu\_exynos / vertex10 / IOMMU
- operação comprovada sem root em grafo NNAPI

### Fontes e variantes

- s5e9945 Device Tree
- referências SM-S721B / U / 0 / Q / J
- kernel SGPU
- firmware e manifests
- inventários ELF e dependências

A NPU é documentada como ramo separado. Ela não é tratada como parte do render node da GPU Xclipse e não altera a conclusão sobre execução SGPU.

### 3. Identidade do dispositivo e do acelerador gráfico

| Campo         | Valor observado             | Classe     |
|---------------|-----------------------------|------------|
| Modelo        | SM-S721B / r12s             | Confirmado |
| Plataforma    | erd9945 / s5e9945           | Confirmado |
| GPU           | Xclipse 940 / MGFx 147      | Confirmado |
| Device ID     | 0x000073a0                  | Confirmado |
| Revisão       | 0x02600200                  | Confirmado |
| GFX IP        | 10.0 / ring mask 0xf        | Confirmado |
| COMPUTE IP    | 10.0 / mask 0x7             | Parcial    |
| Firmware SGPU | 2.23.0                      | Confirmado |
| RTL           | 0x0004ea15                  | Confirmado |
| Nó principal  | sgpu@22200000               | Confirmado |
| Compatible    | samsung-sgpu , samsung-sgpu | Confirmado |

O nome Xclipse 940 identifica o produto e a família observada. Ele não fornece sozinho a codificação ISA, o formato de IB, a compatibilidade com upstream AMDGPU ou a possibilidade de usar um backend Vulkan genérico.

## 4. Device Tree e recursos de plataforma

O nó `/sys/firmware/devicetree/base/sgpu@22200000` possui filhos `gpu_pm`, `gpu_doorbell`, `gpu_debug`, `gpu_smntarg` e `gpu_sysreg`. Os nomes de regiões observados incluem `gpu`, `doorbell`, `debug`, `pwrctl`, `sysreg` e `htu`.

As interrupções são nomeadas `SGPU` e `GPU-AFM`. O nó referencia o domínio `pd_g3dcore@0` / `pd_g3dcore` pelo phandle `0xc3`. Também aparecem propriedades de frequência, DVFS, IFPO, thermal G3D, memória reservada `sgpu_rmem`, `gpu_buffer_dma_heap`, BTS G3D e grupos SysMMU/IOMMU.

A Device Tree representa o contrato de plataforma. Ela informa recursos, dependências, domínios e conexões. Não informa sozinha a semântica integral dos registradores, a ABI de userspace, o formato dos pacotes ou a ordem segura de inicialização de todos os blocos.

## 5. Domínio de energia, clock, thermal e reset

O bloco G3D depende de `G3DCORE` e de seus clocks e resets associados. IFPO e AFM aparecem como elementos de controle e observabilidade. A camada térmica está vinculada ao domínio gráfico e pode alterar frequência, disponibilidade e tempo de conclusão de jobs.

Uma falha em power domain, clock, reset, AFM ou thermal pode se manifestar como falha de MMU, timeout de ring, erro de packet ou ausência de fence. Por isso, esses recursos pertencem à arquitetura de execução, não são apenas metadados de inicialização.

A documentação não afirma que qualquer valor de DVFS ou registrador possa ser escrito com segurança. Os caminhos são tratados como observação de plataforma e como dependências do driver existente.

## 6. Kernel SGPU e DRM

Os caminhos de fonte inventariados incluem:

```
kernel/include/uapi/drm/sgpu_drm.h
kernel/drivers/gpu/drm/samsung/gpu/sgpu/
  amdgpu_drv.c
  amdgpu_cs.c
  amdgpu_vm.c
  amdgpu_gem.c
  amdgpu_object.c
  amdgpu_sched.c
  amdgpu_ring*.c
  amdgpu_ufcode.c
  amdgpu_device.c
  exynos_gpu_interface.c
kernel/drivers/iommu/samsung/
kernel/drivers/dma-buf/heaps/samsung/
```

A nomenclatura AMDGPU-derived aparece na fonte e nos traces, mas isso não autoriza assumir compatibilidade binária ou semântica completa com AMDGPU upstream. A integração Samsung possui nomes, recursos, firmware e vínculos de plataforma próprios.

A UAPI SGPU é o ponto formal de estudo do cliente DRM. Um símbolo interno, um caminho de fonte ou um ioctl aceito não representa uma especificação completa. A estrutura correta é derivar cada campo da relação entre UAPI, kernel, captura e comportamento do dispositivo.

## 7. Topologia DRM observada

A coleta de identificação direta reportou:

```
card0      → amdgpu
renderD128 → amdgpu
card1      → exynos-drmdpu
renderD129 → exynos-drmdpu
```

Essa saída é a camada reportada pelo ioctl. A cadeia física que precisa ser preservada na interpretação é:

nó DRM

- major/minor
- sysfs
- driver associado
- platform device
- Device Tree
- bloco SGPU / G3D

O valor `amdgpu` em um nó não prova uma GPU AMD física, não prova compatibilidade upstream e não substitui a correlação com `sgpu@22200000`. Da mesma forma, `exynos-drmdp` identifica uma camada reportada, não toda a arquitetura de composição ou display.

## 8. GEM, BO, TTM e DMA-BUF

A camada de memória gráfica trabalha com objetos de buffer. Os traces e probes registram criação, movimentação e mapeamento de BO, além de atividades associadas a GEM, TTM e DMA-BUF.

A sequência de recursos é:

cliente DRM

- criação de BO
- escolha de domínio / heap
- mapeamento de CPU ou DMA-BUF
- endereço virtual GPU
- atualização de page tables
- sincronização de VM
- uso em contexto ou submission

Um probe anterior completou criação de BO GTT de 64 KiB, mapeamento CPU, toque de memória, VA map/unmap e fechamento. Isso confirma a infraestrutura de gerenciamento de recurso, mas não confirma que um shader ou comando tenha escrito no BO.

## 9. VM, PTE, PDE e IOMMU

A coleta registra eventos `amdgpu_vm_pte_pde`, `amdgpu_vm_set_ptes`, `amdgpu_vm_update_ptes` e `amdgpu_vm_flush`, além de criação e movimentação de BO.

Em uma janela resumida foram observados 119.929 eventos de `amdgpu_vm_pte_pde` , 11.307 de `amdgpu_vm_set_ptes` , 10.998 de `amdgpu_vm_update_ptes` , 1.947 flushes e 262 criações e movimentos de BO.

A relação operacional observada é:

```
BO / DMA-BUF
  → mapeamento SGPU / PIO
  → amdgpu_vm_bo_cs
  → atualização PTE/PDE
  → amdgpu_vm_flush
  → submissão
  → execução no ring
  → fence / retire
  → limpeza / readback
```

A presença de endereços de page directory, PTEs e flags não revela por si só política de cache, residency, coerência, permissões de cada bit ou recuperação de page fault. O IOMMU é parte da proteção e tradução, não um substituto para a semântica do comando.

## 10. Contexto, BO\_LIST e command submission

O caminho de submission envolve contexto, lista de objetos, chunks e command submission. Os traces registram `amdgpu_cs_ioctl` , `amdgpu_sched_run_job` e `amdgpu_ib_schedule` no ring `gfx_0.0.0` , com `num_ibs=3` .

A cadeia observada pode ser representada assim:

```
contexto
  → BO_LIST / recursos
  → CS ioctl
  → chunks
  → IB(s)
  → scheduler
  → ring GFX
  → command processor
```

Em uma coleta de reprodução, um `DRM_IOCTL_AMDGPU_CS` foi aceito para um chunk IB com `chunk_id=0x1` , VA `0x4000000000` e `ib_bytes=4` . Isso prova aceitação de uma entrada

pelo KMD. Não prova interpretação do IB, execução, write em GPU, fence própria ou readback. Variantes próximas retornaram `EINVAL`, `EFAULT` ou `Bad address`. Essas falhas são úteis para delimitar o contrato, mas não devem ser generalizadas como falha de todo o caminho.

## 11. Scheduler, rings, IB e doorbell

O scheduler organiza jobs e dependências antes da emissão no ring. O ring observado foi `gfx_0.0.0`, e a janela P4.12 registrou 13 ocorrências de `amdgpu_cs_ioctl`, 13 de `amdgpu_sched_run_job` e 13 de `amdgpu_ib_schedule`.

`num_ibs=3` indica que o job observado possuía três IBs na estrutura registrada. Não informa sozinho o conteúdo dos IBs, a função de cada IB ou se todos foram executados.

O nó `gpu_doorbell` e a região `doorbell` aparecem na Device Tree. Um doorbell é uma ponte entre memória/registro e notificação de hardware, mas a documentação não atribui uma codificação específica sem uma prova de escrita e efeito controlado.

## 12. GFX, COMPUTE e SDMA

A identidade de plataforma reporta GFX IP 10.0 com ring mask `0xf` e COMPUTE IP 10.0 com mask `0x7`. O kernel inventariado contém caminhos para rings, firmware, device, GFX e MMHUB.

A atividade GFX foi observada no caminho Android suportado. O caminho COMPUTE possui superfície instalada, exports OpenCL e símbolos de compilador, mas a coleta P5.04 não capturou um dispatch compute, IB compute, scheduler compute ou readback controlado.

SDMA aparece como bloco de infraestrutura em caminhos de firmware e recursos, mas não há neste conjunto uma prova independente de uma transferência SDMA controlada. A classificação correta é presença estrutural e superfície de driver, não execução demonstrada.

## 13. Firmware, microcódigo e recuperação

Os caminhos do kernel incluem `amdgpu_ufirmware.c` e rotinas de carregamento e reset. A arquitetura depende de firmware SGPU reportado como `2.23.0`, da inicialização do domínio gráfico e de recursos de memória e MMU.

Firmware não é tratado como simples arquivo substituível. Versão, revisão RTL, Device Tree, kernel, clocks, reset e estruturas de command processor formam um conjunto acoplado. A presença de um blob ou de uma string de versão não prova compatibilidade com outra revisão.

A existência de caminhos de reset e recovery indica que o driver prevê timeout, erro de job e restauração do bloco. Os traces disponíveis não constituem uma especificação completa da



sequência de recuperação.

## 14. Fences, syncobjs e conclusão

Fences, syncobjs, dependências e retire pertencem à camada de conclusão. Os traces mostram scheduler, IB e dependências; os eventos clássicos de fence não estavam completos em todas as janelas.

A condição de execução válida é:

```
job aceito
  ≠ job executado

job executado
  ≠ resultado correto

resultado correto
  = submissão + conclusão + readback/presentation validado
```

A documentação preserva essa distinção. Um `ioctl_ret=0`, uma fila criada ou um job agendado não é apresentado como execução de shader.

## 15. Shader, ISA e alocação de registradores

O inventário contém símbolos, strings e caminhos relacionados a compiler, shader, ISA e register allocation. Esses sinais ajudam a organizar a arquitetura em compilador, representação intermediária, alocação, emissão e execução.

Não há neste conjunto uma correlação completa entre um shader de entrada, o binário emitido, o packet enviado, a instrução executada e uma saída de memória validada. Por isso, shader/ISA permanecem como indício técnico ou hipótese, não como especificação decodificada.

Wave execution e register allocation são camadas diferentes. A primeira trata de como unidades de execução processam lanes e estados. A segunda trata de recursos e ocupação. Nenhuma delas pode ser inferida apenas pelo nome Xclipse ou pela presença de exports do compiler.

## 16. Texturas, imagens e rendering pipeline

A pilha Android possui Gralloc, SurfaceFlinger, RenderThread e bibliotecas vendor que lidam com imagens e superfícies. Esses consumidores podem utilizar buffers protegidos, formatos específicos, sincronização implícita ou explícita e caminhos de composição que não são equivalentes a um cliente compute.

A arquitetura gráfica distingue:

```
aplicação
→ RenderThread / compositor
→ Gralloc / buffer allocation
→ Vulkan ou OpenGL/driver vendor
→ libdrm / render node
→ SGPU / display pipeline
```

A presença de uma superfície renderizada não demonstra que um buffer de laboratório possa ser reutilizado por um cliente externo. Formato, tiling, cache, fences e ownership são parte do contrato.

## 17. Android, loader e namespaces

O caminho observado em processos suportados é:

```
SurfaceFlinger / RenderThread / câmera
→ loader do sistema
→ /system/lib64/libvulkan.so
→ /vendor/lib64/hw/vulkan.samsung.so
→ libdrm_sgpu.so
→ renderD128
→ kernel SGPU
```

O ambiente Termux está em namespace diferente e expõe `llvmpipe`. O carregamento direto do ICD vendor foi bloqueado por namespaces, dependências, permissões ou combinação desses fatores.

A presença de `vulkan.samsung.so`, `libOpenCL.so` ou `libSGPUOpenCL.so` em `/vendor` não significa que um processo externo possa carregá-los. O namespace Android, o

manifest, o HAL e o domínio SELinux são elementos da arquitetura de acesso.

## 18. Vulkan e OpenCL

A presença de `vułkan.samsung.so` confirma uma superfície ICD vendor instalada. Ela não prova compatibilidade com uma implementação Vulkan de referência nem expõe automaticamente uma ABI para um loader novo.

A presença de `lib0penCL.so` e `libSGPU0penCL.so`, com exports como `clEnqueueNDRangeKernel`, `clEnqueueReadBuffer`, `clBuildProgram` e `clCreateProgramWithIL`, confirma uma superfície OpenCL proprietária. A execução compute externa não foi demonstrada no conjunto P5.04.

A arquitetura de estudo separa três estados:

| Superfície                                 | Estado                             |
|--------------------------------------------|------------------------------------|
| Vulkan vendor dentro do processo suportado | Presença e carregamento observados |
| OpenCL vendor e exports                    | Presença e superfície instaladas   |
| Dispatch compute externo com readback      | Não demonstrado                    |

## 19. Segurança e fronteiras

As fronteiras relevantes são:

- permissões de `/dev/dri/renderD128` ;
- domínio SELinux do cliente;
- linker namespace e dependências vendor;
- manifest e HAL Android;
- storage compartilhado com `noexec` ;
- buffers protegidos e memória reservada;
- separação entre usuário, sistema e vendor;
- recuperação e reset de hardware;
- versão de firmware e revisão de Device Tree.

Root amplia observabilidade, mas não transforma automaticamente a pilha vendor em API pública. `setenforce 0` temporário é um instrumento de diagnóstico, não uma arquitetura de

produção.

## 20. Variantes de plataforma

O inventário contém referências a SM-S721B, SM-S721U, SM-S7210, SM-S721Q e SM-S721J. A existência de arquivos de variantes não permite transportar propriedades sem comparação.

As propriedades que precisam permanecer separadas entre variantes incluem compatible, revision, power-domain, clock, IRQ, IOMMU, DMA heap, firmware, ring mask, compute mask, thermal e recursos reservados.

Uma semelhança de nome de arquivo não constitui identidade de hardware. A matriz de variantes é uma dimensão arquitetural própria.

## 21. Ramo NPU Exynos / NNAPI / ENN

O ramo NPU é separado da GPU Xclipse. A plataforma observada possui

`android.hardware.neuralnetworks.IDevice/enn`, HAL `android.hardware.neuralnetworks-service-enn`, bibliotecas ENN, compiler NPU, Device Tree `npu_exynos`, driver `exynos-npu` e endpoint `/dev/vertex10`.

A rota NNAPI pública foi executada sem root em um grafo `SOFTMAX`. O processo encontrou `enn` e `nnapi-reference`, finalizou modelo, compilou, executou e obteve diferença máxima `0.000000` no readback.

A rota vendor direta `IEnnInterfaceAidl/default` retornou nulo nas rodadas válidas. O endpoint `vertex10` retornou `Permission denied` para UID comum. Com root e permissive temporário, abertura e mmap funcionaram, mas ioctl's tentados retornaram `EINVAL` ou `EFAULT`.

A NPU possui seu próprio conjunto de memória, IOMMU, DVFS, governors, AFM, wakeup, command data e modelos NNC. Esses componentes não devem ser tratados como parte do render node ou da UAPI SGPU.

A arquitetura NPU detalhada está no documento [Mapa detalhado de evidências da NPU Exynos 2400](#).

## 22. Ledger arquitetural

| Camada                          | Estado atual                                | Limite                                                  |
|---------------------------------|---------------------------------------------|---------------------------------------------------------|
| Device Tree SGPU                | Confirmada em grande parte                  | Sem semântica completa de registradores                 |
| Power/clock/thermal/reset       | Recursos e relações observados              | Sequência integral não especificada                     |
| DRM node e identidade reportada | Confirmados                                 | Correlação física completa ainda exige sysfs            |
| GEM/BO/TTM/DMA-BUF              | Atividade observada                         | GPU write controlado não demonstrado                    |
| VM/PTE/PDE/IOMMU                | Atividade observada                         | Cache, residency e fault recovery incompletos           |
| CS/scheduler/IB/ring            | Submissão observada no caminho vendor       | IB próprio e conclusão isolada não demonstrados         |
| Fence/syncobj/retire            | Parcial                                     | Fence de cliente controlado não fechada                 |
| Firmware/recovery               | Caminhos e versão observados                | Contrato completo não publicado                         |
| Vulkan vendor                   | Carregado em processo suportado             | Namespace externo bloqueia reutilização direta          |
| OpenCL vendor                   | Bibliotecas e exports presentes             | Dispatch compute externo não demonstrado                |
| Shader/ISA/compiler             | Indícios estáticos                          | Sem correlação input → ISA → readback                   |
| NPU/NNAPI/ENN                   | Execução <b>SOFTMAX</b> rootless confirmada | Cobertura depende do grafo e da operação                |
| Variantes                       | Referências inventariadas                   | Diferenças de plataforma ainda precisam ser preservadas |

## 23. Limites de publicação

O XOP publica documentação derivada, mapas, hashes selecionados, inventários sanitizados e PDFs. Não publica logs crus, traces integrais, dumps de memória, bibliotecas vendor, binários ELF, firmware, modelos proprietários, credenciais, identificadores pessoais ou scripts que constituam uma tentativa de copiar a pilha Samsung.

A finalidade do documento é tornar a arquitetura compreensível e auditável sem transformar observações parciais em promessas de driver independente.

## 24. Proveniência

A documentação GPU utiliza as coletas de Device Tree, runtime, traces P2/P3/P4/P5, inventários ELF, fontes inventariadas e reprodução DRM recebidas no projeto. O ramo NPU utiliza o pacote `Exynos_NPU.zip` de 18/09/2026, SHA-256

`873a0f8d3ff4d8e9a4168aafab0349f5becfa539cfc9b03d8a7033e3acb08af9` .

Os artefatos brutos permanecem fora do repositório público. Cada conclusão deve ser lida junto com sua classe de evidência e seu limite de interpretação.